

JS高级

布尔教育 <http://www.itbool.com>

燕十八 著

严禁传播 违者必究

高级事件绑定

W3c中的标准--addEventListener

1. 绑在哪个事件上? click,load,change,blur,focus,mouseover,mouseup.....等等
2. 绑定什么函数---自定义事件函数
3. 什么方式监听执行事件函数? 捕捉,冒泡?

语法: xxxDomObject.addEventListener('事件', 函数, 冒泡/捕捉);

注意细节:

- 1: 事件名,一律不带on
- 2: 绑定事件函数中的"this"指绑定该事件的对象
- 3: 执行顺序,是按绑定顺序来执行的!

```
<style type="text/css">
div{
width:400px;
height:400px;
background:blue;
}
</style>
</head>
<body>
<div id="test1">
</div>
</body>
<script>
var test1 = document.getElementById('test1');
test1.addEventListener('click',function() {this.style.backgroundColor='gray'},false);
test1.addEventListener('click',function() {alert('自学');},false);
test1.addEventListener('click',function() {alert('it');}},false);
</script>
</html>
```

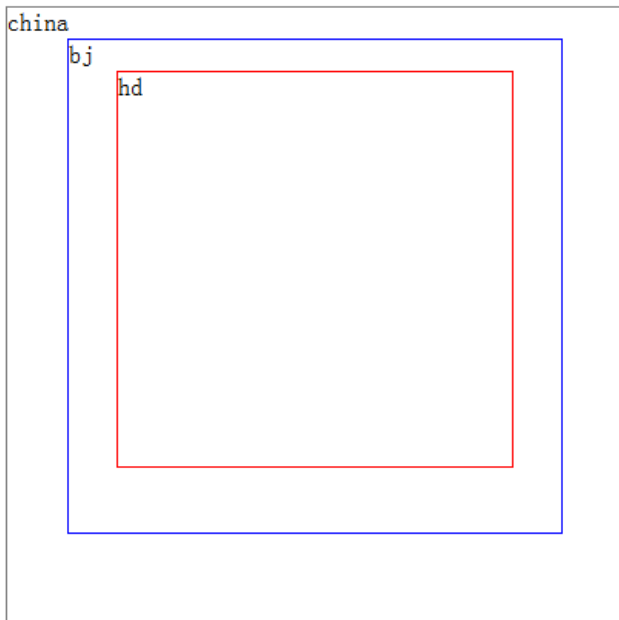
事件之解绑定

与绑定的参数完全一致

removeEventListener()

捕捉与冒泡

思考:如果我在红框hd中,点击一下,china,bj,hd都被点击中了.
而且,如果3者都有click事件,谁先发生?



```
function $(id) {
    return document.getElementById(id);
}

$('china').addEventListener('click', function() {alert('进入china');}, true);
$('bj').addEventListener('click', function() {alert('进入bj');}, true);
$('hd').addEventListener('click', function() {alert('进入hd');}, true);

$('china').addEventListener('click', function(ev) {console.log(ev);alert('离开china');}, false);
$('bj').addEventListener('click', function() {alert('离开bj');}, false);
$('hd').addEventListener('click', function() {alert('离开hd');}, false);
```

停止传播

事件对象有一个方法可以阻止事件的传播：stopPropagation()

例1: 阻止捕捉

```
$('china').addEventListener('click', function(ev) {
    alert('进入china');
    ev.stopPropagation();
}, true);
```

例2: 阻止冒泡

```
$('hd').addEventListener('click', function(ev) {
    alert('进离开');
    ev.stopPropagation();
}, false);
```

阻止事件的默认行为

如阻止一个链接的点击效果,阻止表单的提交效果.

preventDefault()方法

IE下不兼容行为

针对IE模型的事件写法

适用于IE6,7,8

IE的事件模型相比w3c的标准事件,主要有3点不同:

0: 绑定事件的函数不一样, IE中用 attachEvent(),detachEvent

1: 事件必须要加on, 比如标准中第1个参数为click,相对的,IE中要写onclick

2: IE6 模型中,不支持捕捉模型.只支持冒泡(靠JS弥补不了). IE7,8,已经支持捕捉

3: IE7,8中还有个毛病,就是先绑定的事件后发生(准确的说,有可能随机发生)

4: 绑定的事件中, this的指向问题 w3c模型中,this指向对象本身, 而IE模型中this指向window

作用域

```
/*
在JS中，函数嵌套是非常普遍的，
在函数嵌套中，

对变量是如何寻找的？
答：首先在函数内寻找
寻找不到，则往外层函数寻找
...
直到.... 全局(window)区域

*/

var c = 5;

function t1() {
    var d = 6;
    function t2() {
        var e = 7;

        // var d = 3; 注释再去掉注释,观察结果的变化
        alert(c + d + e);
    }

    t2();
}

t1(); // 15, 18
```

var是声明变量

```
/**
var 是在函数运行的上下文中,声明一个变量,
如果不加var,则是一个赋值操作.(按照作用域特点上;溯查找并赋值)

**/

function t1() {
    var d= 5;
    e = 6;

    alert(d);
    alert(e);
}

t1();

console.log(e); // 6
console.log(d); // d is not defined错误
```

作用域测试

```
var str1 = 'global';

function t1() {
    console.log(str1);
    console.log(str2);

    var str2 = 'local';
}

t1(); // ? ?
```

常见的错误是回答"global local"

错误的原因是--不知道JS执行前有一个词法分析过程.

词法分析

词法分析, 分析3样内容

1. 先分析参数
2. 再分析变量声明
3. 分析函数声明

一个函数能使用的局部变量,就从上面的3步分析而来

具体步骤:

1. 函数运行前的1瞬间, 生成 Active Object (活动对象), 下称AO
2. 函数声明的参数, 形成AO的属性, 值为实参的值, 未传实参则为undefined
3. 分析变量声明! 如 var age,
如果AO上还没有age属性, 则添加AO属性, 值是undefined
如果AO上已经有age属性, 则不做任何影响
4. 分析函数声明, 如 function foo() {},
则把函数赋给AO.foo属性
**注: **如果此前foo属性已存在, 则被无情的覆盖了

词法分析练习

第1题:

```
var age = 99;
function t(age) {
    alert(age);
}

t(5); // 5
t();// undefined
```

词法分析过程:

AO {age:5}

运行过程:

alert(AO.age); //5

t() ----> AO.age没得到赋值, 还是undefined

第2题:

```
function t2(age) {
    var age = 99;
    alert(age);
}

t2(5);
```

分析过程:

- 0: 形成AO = {}
- 1: 分析参数 AO = {age:5}
- 2: 分析var age, 发现AO已有age属性, 不做任何影响

执行过程:

执行赋值: AO.age = 99;

alert(age);

第3题:

```
function t3(greet) {
  alert(greet);

  function greet() {
  }

  greet = 'hello';
  alert(greet);
}

t3(null); // function hello
```

分析过程:

0: 形成AO = {}

1: 分析参数 AO = {greet:null}

2: 分析函数声明, AO.greet = function(){} 被覆盖成函数

执行过程:

alert(age);

AO.greet = hello;

alert(age);

第4题

```
// 再看这道题

function a(b) {
  alert(b);
  function b(){
    alert (b);
  }
  b();
}

a(1);
```

分析期:

0: AO = {}

1: 分析参数 AO = {b:1}

2: 分析var 声明,此函数没有var

3: 分析函数声明, AO = {b: function(){alert(b);}}

执行期:

alert(b); // function

b(); // 由作用域寻找到a函数中的b,即 function,alert()出来

第5题:

```
function a(b) {
  alert(b);
  b = function (){
    alert (b);
  }

  b();
}

a(1);
```

学员常见答案:

1,1

1, function

1, undefined

function ,function

词法分析过程:

0: AO = {}

- 1: 分析参数 AO = {b:1}
- 2: 分析var声明,没有.
- 3: 分析函数声明?? 没有!

(注: b = function() {} ,是一个赋值过程,在执行期才有用)

执行过程:

```
alert(b); // 1
b = function() {
  alert(b);
}
b(); // function
```

函数声明与函数表达式

JS中,函数也是一种值,
可以赋值给变量,可以作为参数来传递.

```
function t1() {
}
```

```
t2 = function() {
}
```

这2种方式,效果不同的,

t1是函数声明, 虽然全局内也得到一个t1变量,值是function

而t2 只是一个赋值过程-----值是谁? 值是右侧的表达式的返回结果,即函数

就是说 function () {} 在js看来,就和 3*2, 6/3 一样,是个表达式,返回一个结果

因此,t1 t2 两种方式在词法分析时,有着本质区别

前者 在词法分析阶段,就发挥作用

而后者,在运行阶段,才发挥作用

知道了函数表达式的概念,再看看一个你以前看不懂的东西

```
(function(window,undefined) {
})(window);
```

这是jquery的最外层代码

```
(function(window,undefined){}) // 内层表达式,返回值是函数,包在小括号里,当成表达式来执行\
(function(window,undefined){})(window) // 立即调用
```

而内层函数又没有起名字,称为匿名函数,

这种手法,匿名函数,立即执行,不污染全局. 称为 立即执行匿名函数表达式

问: 为什么传window, 而又不会传undefined?

/*

答: 传window是为了速度

```
function() {
  function() {
    function () {
      function () {
        function () {
          window.document.getElementById... // window将会向作用域层层上找,直到最外层
        }
      }
    }
  }
}
```

jquery就是为了加快内部查找全局变量的速度, 而直接把window以参数形式传进来

这样 window 就在jquery内部的 AO上

不传undefined是为了安全

因为在IE,FF低版本中, undefined竟然可以重新赋值 ,如 undefined = 3;

声明undefined局部变量(名字是undefined而已), 同时,又不传参,值自然是undefined
防止了外界对undefined的污染

arguments对象

函数运行其内,关键的三个对象

AO:本函数AO上没有某属性,则继续去外层函数的AO上找,直到全局对象. 叫做"作用域链"

arguments: 每个函数有自己的arguments,但 不 向外层接着找arguments的相关属性,即 不形成链

this: 沿原型查找,形成原型链

arguments是什么?

答: 是1个对象,是一个长的很像数组的对象

arguments内容是什么?

答: arguments是函数调用时的实参列表.

代码实验:

```
(function (d,e,f) {  
    console.log(arguments); // 对象  
    console.log(arguments[0]) // hello -->长的像数组  
    console.log(arguments[2]) // !  
})('hello', 'world', '!');
```

arguments代表收到的所有实参

```
(function (d,e,f) {  
    // 在此函数内,无法用d,e,f形参来取得'haha',因为没有与之相应的形参  
    // 但我们可以用arguments来获取任意多个的实参  
    console.log(arguments[3]);  
  
})('hello', 'world', '!', 'haha');
```

arguments.callee 属性

callee 代表 "当前运行的函数"

```
(function (d,e,f) {  
    console.log(arguments.callee);  
})('hello', 'world', '!');
```

this是谁

js中的this灵活多变,极易出错.

可以按照函数的4种调用方式分别讨论

1:作为普通函数来调用时,

this的值指向->window

准确的说,this为null,但被解释成window

在ECMAScript5标准中, 如果this为null,则会抛出异常

```
function t() {  
    this.xx = 333;  
}  
  
t();  
  
alert(window.xx);
```

2: 作为对象的方法来调用

作为方法调用时,this指向其调用那一瞬间的调用者,即调用对象

而不论函数声明时,this属于谁.

```
show = function() {  
    alert('show '+this.xx);  
}
```

```
dog.t = show;
dog.t();// show wangwang
```

3: 作为构造函数调用时

js中没有类的概念, 创建对象是用构造函数来完成, 或者直接用json格式{}来写对象

```
function Dog(name,age) {
    this.name = name;
    this.age = age;
    this.bark = function() {
        alert('this is ' + this.name + '!');
    }
}

var dog = new Dog('huzi',2);
dog.bark();
```

new Dog发生了以下几个步骤

- a: 系统创建空对象 {}, (空对象constructor属性指向Dog函数, 先别管)
- b: 把函数的this---指向-->该空对象
- c: 执行该函数
- d: 返回该对象

下面这个返回什么?

```
function Pig() {
    this.age = 99;
    return 'abc';
}
```

var pig = new Pig(); // Pig对象, 因为函数作为构造函数运行时, return 的值是忽略的, 还是返回对象.
console.log(pig);

4: 函数通过call, apply调用

语法格式: 函数.call(对象, 参数1, 参数2...参数N);

```
function t(num) {
    alert('我的真实年龄是' + this.age);
    alert('但我一般告诉别人我' + (this.age + num));
}

var human = {name:'lisi', age:28};

human.t = t;
human.t(-10); // this指向了human, 但是human多了一个方法.

// 接下来, 我们不把t赋为human的属性, 也能把this指向human
var wangwu = {name:'wangwu', age:30};
t.call(wangwu, 5);
```

解释

fn.call(对象obj, 参数1, 参数2...参数N);

运行如下:

- a): fn函数中的this----指向-->对象obj
- b): 运行fn(参数1, 参数2...参数N);

this练习题:

```
name = 'this is window';

var obj = {name:'php', t:function () {alert(this.name)}};
var dog = {name:'huzi'};
```

```
obj.t(); // php

var tmp = obj.t;
tmp(); //相当于window.tmp() this is window

dog.t = obj.t;
dog.t(); // huzi;

(dog.t = obj.t)();
// (dog.t = obj.t)是一个表达式,返回值是"值",即t函数,
// 强调是值,说明不是通过引用来调用的,而是立即使用函数本身
// 效果等同于 (function () {alert(this.name)}})()
// null又被解释成window

dog.t.call(obj); //php
```

QQ群问题:

天(372XXX95) 9:04:17

```
var name = "The Window";
var object = {
  name : "My Object",
  getNameFunc : function(){
    return function(){
      return this.name;
    };
  }
};
alert(object.getNameFunc()());
请教各位大神.....为什么是The window
```

闭包

```
function t1() {
  var age = 20;

  function t2() {
    alert(age);
  }

  return t2;
}

var age = 99;
var tmp = t1();
tmp(); // 20
```

:在大部分的语言中,t1被调用执行,则申请内存
并把其局部变量,push入栈!

t1函数执行完毕,内部的局部变量,随着函数的退出而销毁
因此, age = 20的局部变量,依靠已经消失了.

但是在js中, age = 20这个变量,却被t2捕捉,
即使t1执行完毕,通过t2,依然能访问该变量

这种情况---返回的函数,并非孤立的函数,甚至把其周围的变量环境,
形成了一封闭的"环境包",共同返回, 所以叫"闭包"

----一句话概括-----函数的作用域取决于声明时,而不取决于调用时!

闭包计数器

多个人开发js程序, 需要一个全局的计数器

--多个人的函数共用一个计数器,计数器一直增长

解决办法:

1: 设立一个全局变量

```
window.cnt = 0;
```

调用 ++window.cnt;

这个办法可行,

但是,污染了全部变量.

其次,引入了多人的js程序,别人的程序里,也有一个window.cnt = 'hello';

该计数器就损坏了.(所以要尽量避免用全局变量)

2: 闭包维护一个别人污染不到的变量,做计数器

```
function counter() {  
    var cnt = 0; // 当counter执行完毕后,除了返回的cnter函数,谁也别想碰到cnt变量了。  
  
    var cnter = function () {  
        return ++cnt;  
    }  
  
    return cnter;  
}  
  
var inc = counter();  
  
alert(cnt()); // 1  
alert(cnt()); // 2  
alert(cnt()); // 3
```

// 第2版 匿名函数简化

```
var cnt = (function () {  
    var cnt = 0;  
    return function() {  
        return ++cnt;  
    }  
})();  
  
alert(cnt());  
alert(cnt());  
alert(cnt());
```

第3版 如何避免全局污染或冲突

/*

1:统一放在一个全局对象上,如jquery->\$

2:每人用自己的命名空间.

*/

// jquery的计数器插件形式

```
$ = {};  
$.cnt = (function () {  
    var cnt = 0;  
    return function() {  
        return ++cnt;  
    }  
})();  
  
alert($.cnt());  
alert($.cnt());  
alert($.cnt());
```

// 个人命名空间,在团队开发中也很常见

// 其实就是把自己的变量,函数,都放在一个对象里.

```
var Y18 = {}; // 跟团队说清,这是我的空间,别人不要用了  
Y18.cnt = (function () {  
    var cnt = 0;
```

```
        return function() {
            return ++cnt;
        }
    })();

    alert(Y18.cnt());
    alert(Y18.cnt());
```

JS面向对象

先有人 还是 先有类?

如果从造物主的角度讲, 脑中,先构想"人类的形象",然后再造人
先有类,再有对象. 如java,php,c++,先把类构思好,才能对象.

如果从地球生物演化的角度讲,
先有人,才有类.

因为,原本没有人,只有猴子.

一个雷打下来, 某个猴子变异了,能站着走路,脑子也聪明了-----

总结这个猴子及其后代---->和普通猴不一样,把他们叫人类.

其次: 在java,PHP中,通过类造的对象,所有的都一个模板,比如 能哭,会笑
如果类改变了,所有对象的方法,都改变.

这也不符合生物的规律

因为,对象是以个体为单位的, cat1,cat2, 一个会爬树,一个不会.

这个java,php中如何实现?

在而js中, 就很自然

```
var ori = {}; // 地球上最原始的蛋白质, 这就是一个对象
var c = {cell:1}; // 单细胞动物

var bird = {
    leg:2,
    song:function() {alert('我是一只小小鸟');}
}

console.log(ori,c,bird);

bird.song();
```

js的对象的属性,可以任意增添和删除的

```
bird.wine = 2; // 增加2条翅膀
console.log(bird.wine);

// 这只小鸡受到弹弓击打,不会唱歌了!
delete bird.song;
console.log(bird);
```

构造函数

```
function Dog(){
    this.leg = 4;
    this.bark = function() {
        alert('汪汪');
    }
}

var huzi = new Dog();
huzi.bark();

alert(huzi.leg);
```

此处, 我们并没有完成面向对象的 " 封装 "

所谓封装--就要封闭一部分,外界无法访问,
开放一部分,通过开放部分间接访问私有部分.

接下-----js之私有属性!

私有属性

```
// 用闭包来完成js面向对象之私有属性
// 让love不能外界访问

function Girl (name,bf) {
    var love;

    this.name = name;

    // 通过showlove做接口, 来读取私有属性love
    this.showlove = function() {
        return love;
    }

    // 移情别恋
    this.movelove = function() {
        love = '薛潘';
    }
}

var girl = new Girl('林黛玉','贾宝玉');
alert(girl.name + '喜欢' + girl.showlove());

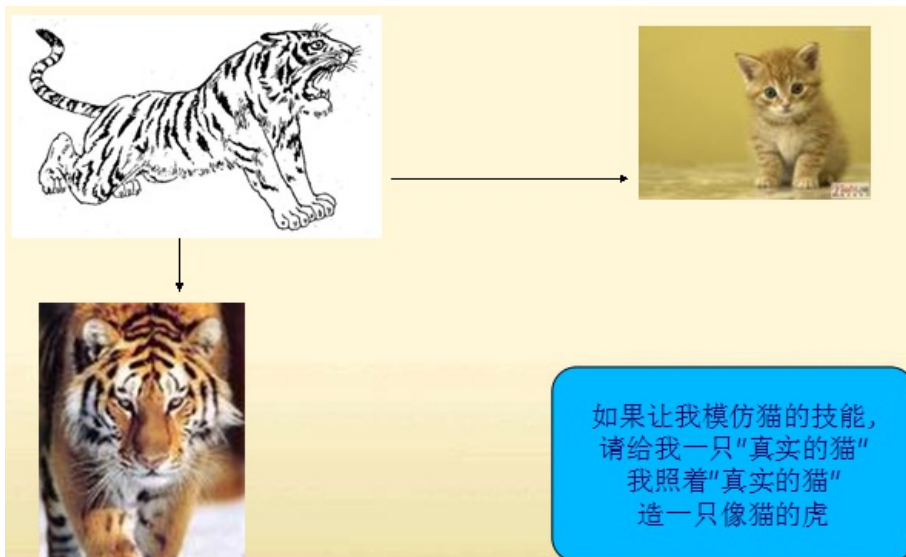
girl.movelove();
alert(girl.name + '喜欢' + girl.showlove());
```

这是通过闭包来完成js面向对象的私有属性与封装

这是方法是JS大牛, Douglas Crockford 提出来的,
他以前在Yahoo,现在在paypal,

JSON格式也是他倡导的,
他的书---the Javascript good parts;

原型



```
function Cat() {
    this.climb = function() {
        alert('我会爬树');
    }
}
```

```
function Tiger() {
    this.bark = function() {
        alert('我是百兽之王');
    }
}

Tiger.prototype = new cat(); // 用1只猫做"造虎机"的原型
var hnhu = new Tiger();
hnhu.climb();

alert(hnhu.valueOf()); // 正常调用
console.log(hnhu);
```

在这个过程中,发生了什么?

- 1: 构造空对象hnhu {}
- 2: hnhu.bark = function () {}
- 3: hnhu.**proto** = Tiger.prototype(即cat对象) // 这是继承的关键

就是说---js中,每个对象,都有一个__proto__指向其原型对象
原型对象也是对象,也有__proto__

..原型的原型的原型的原型.....是谁?

```
hnhu.__proto__
hnhu.__proto__.__proto__
hnhu.__proto__.__proto__.__proto__
```

修改原型

```
// 给所有的对象,加一个唱歌方法

Object.prototype.sing = function() {
    alert('出卖我的爱');
}

function Pig() {
    this.eat = '30KG'; //每次吃30千克
}

var zhu = new Pig();

zhu.sing();
```

原型冒充

JS的语法,非常灵活

不仅可以用原型继承,还有其它办法

如,原型冒充,或复制继承

```
function Cat(leg,tail) {
    this.leg = leg;
    this.tail = tail;

    this.climb = function() {
        alert('我会爬树');
    }
}

function Tiger(leg,tail,color) {
    // 把要继承的类的语句,拿来执行一遍
    this.parent = Cat; // 把父类构造函数引入到一个parent属性上
    this.parent.apply(this,arguments);

    delete this.parent;
```

```
    this.color = color;
}

var tiger = new Tiger(4,1,'yellow'); //

console.log(tiger);

tiger.climb();
```

这是什么过程？

答:其实就是,在用Tiger造对象时,用Tiger的语句影响一个空对象{},
在此过程员,Tiger影响空对象前,先由Cat函数实施影响

因此,最终得到的对象,是由Cat和Tiger两者共同作用过的对象

复制继承

```
// 复制继承,就是把父对象的所有属性,直接复制到自己对象上
function Cat(leg,tail) {
    this.leg = leg;
    this.tail = tail;

    this.climb = function() {
        alert('我会爬树');
    }
}

function Tiger(color) {
    this.color = color;

    this.extend = function (parent) {
        for(var key in parent) {
            //console.log(key);
            this[key] = parent[key];
        }
    }
}

var tiger = new Tiger('yellow');
//tiger.climb(); 出错
console.log(tiger);

tiger.extend(new Cat(4,1));
console.log(tiger);

tiger.climb();
```

函数也是对象

```
// 字符串是对象
var str = 'hello';
str.substr(2,3);

// 函数也是对象
var func = function (num1 , num2) {
    return num1 + num2;
}

// 是对象当然有属性
console.log(func.length); // 2 即定义函数时形参的数量

// 添加1个属性
func.ajax = function() {
    alert('发ajax请求');
}
```

jQuery插件

jQuery插件其实就是在jQuery对象上增加方法.
jQuery通过extend把对象的方法复制继承过去.